MVP WALKTHROUGH · FOR ENGINEERING

Vigio

Case intelligence for pharmacovigilance: from a raw adverse event report to a reviewer-ready brief in under a minute.

Abhishek Pandey, CEO · Prepared for Surya Sunkavalli, CTO · October 2026

THE PROBLEM

Every adverse event case needs four judgments, made by hand

01

Is it valid?

Confirm the four ICH E2A elements: patient, reporter, suspect drug, event.

02

How serious?

Apply 21 CFR 312.32 criteria, which set a 7- or 15-day reporting clock.

03

Did the drug cause it?

Weigh timing, history, other medications, other causes and the literature.

04

Is it a signal?

Check FDA FAERS data for disproportionate reporting across patients.

Today a safety reviewer does this across several tools, under a regulatory deadline, case after case.

WHAT VIGIO DOES

One PDF in, one brief out

INPUT

A raw case report

An Individual Case Safety Report (ICSR) as a PDF: a MedWatch or CIOMS form, or a physician narrative. Unstructured, inconsistent, written for humans.



OUTPUT

A Case Intelligence Brief

- Validity, seriousness, reporting deadline
- Literature evidence with PubMed citations
- FAERS signal strength and confidence
- Recommended action, every claim sourced

Design rule: Vigio builds the evidence; a qualified reviewer confirms the determination. Every brief ends in a human sign-off.

THE FRAMEWORK

Three lenses on every case

LENS 1

Regulatory Safety

Is the case valid, serious, expected and reportable?

Intake, expedited and aggregate reporting, labeling, compliance

LENS 2

Clinical Safety

What happened to the patient, and how likely is the drug the cause?

Medical review, causality, time-to-onset, dechallenge, risk factors

LENS 3

Population Safety and Evidence Generation

With other cases and data, is there a risk worth investigating?

Signal detection, observed vs. expected, real-world evidence, studies

These are not sequential steps: each lens asks a different question of the same case, and every brief answers all three.

THE PIPELINE

Five stages from upload to sign-off



The frontend polls /api/status for progress (0–100%) while the backend runs stages 1–4, then renders the brief for stage 5.

STAGE 3 IN DETAIL

Each lens: built today vs. the seed demo

Lens	Built today	Adding for the seed demo
Regulatory	Validity, seriousness, 7- or 15-day trigger	Expectedness against the drug label; saved audit trail
Clinical	Literature mechanism with PubMed citations	Time-to-onset, dose, dechallenge, risk factors, other causes; WHO-UMC causality; follow-up questions
Population and evidence	FAERS reporting odds ratio with data confidence	Evidence-gap statement; FAERS breakdown by sex and age (stretch)

WHAT YOU CAN CLICK TODAY

A clickable demo with three realistic cases

Demo cases

Warfarin · intracranial hemorrhage

Pembrolizumab · immune myocarditis

Metformin · lactic acidosis

Tabs in each brief

1. **Source Case:** the raw ICSR as received
2. **Summary:** verdict and recommended action
3. **Regulatory, Clinical, Population:** one per layer
4. **Confirm:** reviewer sign-off and audit trail

The demo’s cases are hardcoded in standalone.html. The backend that runs real uploads is written but not yet deployed.

TECH STACK

Python backend, Claude for reasoning, a light frontend

Backend

FastAPI, Uvicorn, Pydantic v2, PyMuPDF, httpx

AI

Anthropic SDK; model set in each service file

Frontend

standalone.html (React 18 via CDN, no build), served at /ui. A React + TypeScript + Vite version sits in frontend/src.

Method	Endpoint	Purpose
POST	/api/upload	Upload PDF, get case_id
POST	/api/analyze	Start the pipeline
GET	/api/status/{case_id}	Poll progress
GET	/api/brief/{case_id}	Fetch the brief
POST	/api/confirm/{case_id}	Reviewer decision
GET	/api/export/{case_id}	JSON or E2B export

THE CODEBASE

Where everything lives

```
vigio-mvp/
├── backend/
│   ├── main.py
│   ├── routers/cases.py
│   ├── models/case.py
│   └── services/
│       ├── pdf_parser.py
│       ├── entity_extractor.py
│       ├── regulatory_intelligence.py
│       ├── clinical_intelligence.py
│       ├── population_intelligence.py
│       └── brief_synthesizer.py
├── frontend/standalone.html
└── requirements.txt, .env.example
```

Read these first

`routers/cases.py`

The API, and the function that runs the whole pipeline

`models/case.py`

The data model for every layer and the brief

`services/`

One file per stage, including the Claude prompts

KNOWN GAPS

What the MVP does not do yet

Gap	What it means	Likely fix
Not hosted	Demo is local or uses hardcoded cases	Deploy backend and UI at a URL
In-memory storage	Cases are lost on restart	Add a database
Text-layer PDFs only	Scanned forms return no text	Add an OCR step
Layers run in sequence	Slower than it needs to be	Run the three layers concurrently
Expectedness unknown	No product label loaded	Ingest labels and brochures
E2B export is a stub	Not submission-ready	Full E2B(R3) mapping

PROPOSED PRIORITIES

Goal: process real cases live in front of investors

For the seed demo

1. Host the backend and upload UI at a URL
2. Upload and process multiple cases at once
3. Run real MedWatch and CIOMS PDFs end to end
4. Save cases and the audit trail in a database
5. Add OCR for scanned reports
6. Clinical review, causality, expectedness
7. Follow-up questions and evidence gaps

After funding

- Aggregate analysis of a customer’s cases
- Observed vs. expected, real-world evidence
- E2B(R3) export, Argus and Veeva links
- Validation, compliance, enterprise security

WHERE THIS GOES

From one case to evidence for regulators

1. **One ICSR assessed through all three lenses**
2. Thousands of a customer's cases analyzed together
3. A pattern flagged as a potential signal
4. The evidence gap behind it identified
5. Claims, EHR, registry data or a study fill the gap
6. Incidence and risk characterized
7. Evidence informs benefit-risk and regulatory decisions

WHERE THE MVP SITS

The MVP is step 1. The evidence-gap statement in every brief already points toward step 4.

GETTING STARTED

Run it locally in four commands

```
$ cp .env.example .env # add ANTHROPIC_API_KEY
$ pip install -r requirements.txt
$ python -m backend.main
$ open http://localhost:8000/ui
```

Next step: a walkthrough together, then agree the build plan for the seed demo.